

Algorithms for Data Science

Lecture #7: Dictionaries and Data Structures

Mateusz Buczynski

University of Warsaw

29.04.2026

Thanks to dr Krzysztof Fleszar for the base material.

Why Data Structures Matter

A data structure is more than storage

It stores data *and* maintains an invariant that makes some operations efficient.

Typical question

Which operations do we need to be fast: lookup, insertion, deletion, minimum, connectivity, approximate membership?

Dictionary: Abstract Data Type

Core object

A dictionary stores unique **key-value pairs**.

Main operations

- `Init()`
- `Check(k)`
- `Get(k)`
- `Add(k, v)`
- `Remove(k)`

Design goal

Choose a representation that makes the expensive operations cheap in the workload we actually care about.

A Running Theme: Trade-offs

Fast lookup

Often needs more structure.

Fast insertion

Often prefers looser structure.

Low memory

May hurt operation time.

Lecture viewpoint

We compare structures by operation cost, memory cost, and use case.

Direct Addressing

Assumption

Keys are integers in a known range:

$$k \in [0, M].$$

Representation

Store values in an array $a[0..M]$, where index = key.

Main idea

The key itself tells us where to look.

Direct Addressing: Costs

Operations

Init =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) = $\Theta(1)$

Add(k, v) =

Direct Addressing: Costs

Operations

Init = $\Theta(M)$

Check(k) = $\Theta(1)$

Get(k) = $\Theta(1)$

Add(k, v) = $\Theta(1)$

Space

$\Theta(M)$

When it is good

Excellent when the key range is small and dense.

Direct Addressing: Limitation

Sparse key spaces are expensive

If keys range up to one billion but only a thousand are present, direct addressing wastes memory badly.

Lesson

Perfect lookup time can be too expensive in space.

Unsorted Array of Key-Value Pairs

Representation

Store entries as a list of pairs:

$$[(k_1, v_1), (k_2, v_2), \dots, (k_n, v_n)]$$

Effect

Very simple, memory-efficient, but lookup requires scanning.

Unsorted Array: Operation Costs

Operations

Init =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(n)$

Get(k) =

Unsorted Array: Operation Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(n)$

Get(k) = $\Theta(n)$

Add(k, v) =

Unsorted Array: Operation Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

in the general dictionary view, because we must first check whether k already exists.

If the New Key Is Guaranteed to Be New

Special case

If we know in advance that k is not yet present, then we can append (k, v) .

Cost with a dynamic array

$$\text{Add}(k, v) = \Theta(1) \text{ amortized}$$

Important distinction

Worst-case cost of one append can be large because resizing may be needed.

Dynamic Arrays and Resizing

Typical strategy

When the array is full, allocate a new array of double size and copy the old elements.

What happens

Most insertions are cheap; occasionally one insertion triggers a big copy.

Amortized Analysis: The Big Idea

Question

How expensive is one insertion *on average over a long sequence* of insertions?

Answer

Even though some insertions are expensive, the amortized cost stays

$$\Theta(1).$$

Why Amortized Cost Stays Constant

Doubling argument

Copies happen at sizes

$$1, 2, 4, 8, \dots$$

so the total number of copied elements after n insertions is

$$1 + 2 + 4 + \dots < 2n.$$

Conclusion

n insertions cost $O(n)$ total, so one insertion costs $O(1)$ amortized.

Sorted Array Dictionary

Invariant

Keep pairs sorted by key.

Main benefit

We can use binary search for $\text{Check}(k)$ and $\text{Get}(k)$.

Cost we pay

Keeping the order makes insertion harder.

Sorted Array: Costs

Operations

Init =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(\log n)$

Get(k) =

Sorted Array: Costs

Operations

Init = $\Theta(1)$

Check(k) = $\Theta(\log n)$

Get(k) = $\Theta(\log n)$

Add(k, v) =

Sorted Array: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(\log n)$
Get(k)	=	$\Theta(\log n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

because elements may need to be shifted.

Sorted Linked List Dictionary

What it is

Store key-value pairs in a linked list sorted by key:

$$(k_1, v_1) \rightarrow (k_2, v_2) \rightarrow \cdots \rightarrow (k_n, v_n)$$

with $k_1 < k_2 < \cdots < k_n$.

How it works

Walk forward from the head until we either find key k or reach the first larger key.

Key difference from a sorted array

The order is maintained, but there is no binary search because we do not have random access.

Sorted Linked List: Array View

Logical order stored in array cells

The nodes can sit in arbitrary array positions; sorting is enforced by the next links.

Before insertion: head = 0, logical order $0 \rightarrow 2 \rightarrow 1 \rightarrow \text{nil}$

index	0	1	2
pair	(1, 10)	(4, 5)	(3, 9)
next	2	NIL	1

Add(2, 7): use free cell 3 and redirect one link

After insertion: head = 0, logical order $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{nil}$

index	0	1	2	3
pair	(1, 10)	(4, 5)	(3, 9)	(2, 7)
next	3	NIL	1	2

Sorted Linked List: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why these costs

We must scan nodes one by one to find the right position, even though the list is sorted.

Linked List Dictionary

What it is

Store key-value pairs in a plain linked list with no order requirement.

How it works

To search, scan node by node until key k is found or the list ends. To add a new pair, insert a new node at the front.

Main tradeoff

Insertion is simple, but lookups still require a full scan in the worst case.

Linked List: Costs

Operations

Init	=	$\Theta(1)$
Check(k)	=	$\Theta(n)$
Get(k)	=	$\Theta(n)$
Add(k, v)	=	$\Theta(n)$

Why Add is linear

In the dictionary view, we still must first check whether k is already present before inserting a new node.

Exact Dictionary Structures So Far

Summary

Structure	Init	Check(k)	Get(k)	Add(k, v)	Space
Direct addressing	$\Theta(M)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(M)$
Unsorted array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Sorted array	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Sorted linked list	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Linked list	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Stack

Rule

Last In, First Out (LIFO).

Operations

- `push(x)`
- `top()`
- `pop()`

Typical uses

Recursion simulation, DFS, undo, parsing.

Queue

Rule

First In, First Out (FIFO).

Operations

- `enqueue(x)`
- `front()`
- `dequeue()`

Typical uses

BFS, simulations, scheduling, stream processing.

Why Stack and Queue Matter

Stack order

Go deep quickly, then come back.

Queue order

Process in arrival order, layer by layer.

Same data, different control flow

The chosen structure can completely change the algorithm behavior.

Priority Queue

Rule

Remove the element with best priority, not the oldest and not the newest.

Typical operations

- `insert(x)`
- `extractMin()` or `extractMax()`
- `decreaseKey()` in some implementations

Priority Queue via a Binary Heap

Heap invariant

In a min-heap, every node key is at most the keys of its children.

Costs

$\text{insert} = O(\log n), \quad \text{extractMin} = O(\log n)$

Why it matters

Dijkstra and many scheduling algorithms rely on priority queues.

Union-Find: Dynamic Connectivity

Goal

Maintain a partition of elements into disjoint sets.

Operations

- `Find(x)`: which set contains x ?
- `Union(x,y)`: merge the sets containing x and y

Typical use

Connectivity queries and Kruskal's MST algorithm.

Union-Find Representation

Forest view

Each set is represented by a rooted tree; the root is the representative.

Core rule

$\text{Find}(x)$ follows parent pointers to the root.

Why Union-Find Is Fast in Practice

Two key heuristics

- union by rank or size
- path compression

Practical result

Operations become almost constant-time for realistic input sizes.

Hash Table Dictionary

Main idea

Use a hash function $h(k)$ to map key k to one bucket in an array of size m .

Representation

Store an array of buckets and put pair (k, v) into bucket $h(k)$.

Why it helps

If keys spread evenly across buckets, each bucket stays short and operations become very fast.

Hash Functions

What a hash function does

A hash function maps a key to an array index:

$$h(k) \in \{0, 1, \dots, m - 1\}.$$

Simple example

For integer keys, one common idea is

$$h(k) = k \bmod m.$$

Example: if $m = 5$, then $h(12) = 2$ and $h(27) = 2$.

What we want

A good hash function spreads keys as evenly as possible, so buckets stay short.

Collisions and Chaining

Collision

Different keys can hash to the same bucket.

Standard fix: chaining

Each bucket stores a short linked list of all pairs whose keys hash there.

bucket 0 : $\emptyset$ bucket 1 : $(1, 10) \rightarrow (9, 4)$ bucket 2 : $(2, 7)$

Search rule

Hash first to find the bucket, then scan only inside that bucket.

Hash Table: Costs

Operations with chaining

Init = $\Theta(m)$
Check(k) = $\Theta(1)$ expected
Get(k) = $\Theta(1)$ expected
Add(k, v) = $\Theta(1)$ expected

Important caveat

Worst case is $\Theta(n)$ if many keys fall into the same bucket.

Assumption behind the fast case

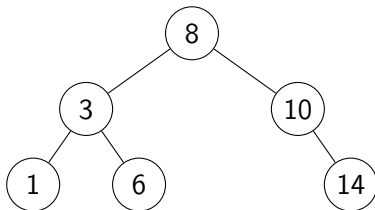
A good hash function and a bounded load factor.

Binary Search Tree (BST)

Invariant

For each node with key k :

- all keys in the left subtree are smaller than k
- all keys in the right subtree are larger than k



BST Search Logic

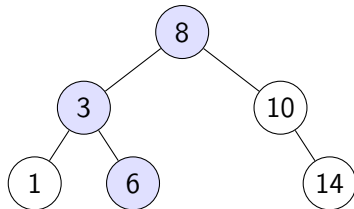
Decision rule

At a node with key k :

- if target $< k$, go left
- if target $> k$, go right
- if target $= k$, stop

Cost

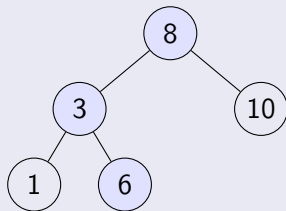
Proportional to the tree height h .



search for key 6: $8 \rightarrow 3 \rightarrow 6$

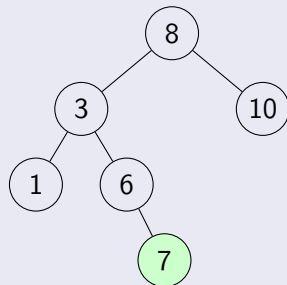
BST Insertion Example

Before adding 7



path for 7: $8 \rightarrow 3 \rightarrow 6$

After adding 7

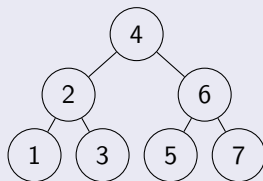


Insertion rule

Follow the BST comparisons until you reach an empty child pointer, then insert there.

BSTs Can Be Excellent or Terrible

Balanced shape



$$h = \Theta(\log n)$$

Degenerate shape



$$h = \Theta(n)$$

Lesson

The same keys can produce very different performance depending on the tree shape.

How a BST Becomes Unbalanced

Insert sorted keys

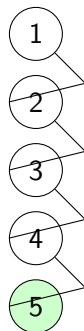
If we insert

1, 2, 3, 4, 5

in this order, each new key becomes the right child of the previous one.

Consequence

The BST behaves like a linked list, so lookup and insertion degrade to linear time.



Self-Balancing BSTs

Idea

Maintain extra structural rules so the height stays logarithmic.

Examples

AVL trees, red-black trees.

Why rebalancing matters

Without it, BST guarantees collapse on unlucky insertion orders.

AVL Trees

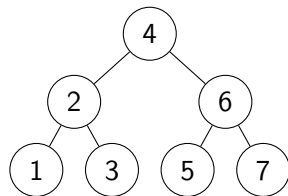
AVL invariant

At every node, the heights of the left and right subtrees differ by at most 1.

Effect

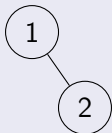
$$h = \Theta(\log n)$$

so search, insertion, and deletion are logarithmic.



AVL Insertion and Rebalancing

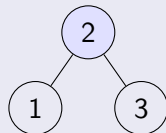
Before add(3)



After add(3)



After left rotation

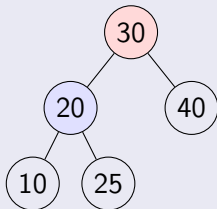


What happened

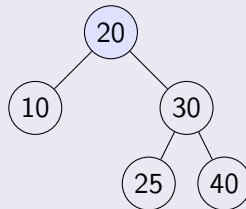
The new key made node 1 too right-heavy, so one local rotation restored balance.

Rotations: Local Repairs

Before right rotation



After right rotation



Key point

Rotations are local changes that preserve the BST order invariant while reducing height.

Exact Structures: Intermediate Summary

What improves from left to right?

array scan $\rightarrow$ binary search / hashing $\rightarrow$ balanced tree

What the structures buy us

More invariants or better key distribution usually mean faster queries.

Why Probabilistic Data Structures?

Big-data motivation

Sometimes exact answers are too expensive in time or memory.

Alternative

Accept a small, controlled probability of error to save massive space.

Bloom Filter: Goal

Task

Support approximate membership queries:

“Have we seen key x before?”

Promise

Very small memory footprint, very fast queries.

Bloom Filter: Structure

Representation

- a bit array of length m
- k hash functions

Insert x

Hash x with all k functions and set the corresponding bits to 1.

Bloom Filter: Query Logic

Check x

If at least one of the k bits is 0, then x is definitely absent.

But if all bits are 1

x is *probably* present.

Error type

False positives are possible; false negatives are not.

When Bloom Filters Are Useful

- web crawling: have we already visited this URL?
- databases: avoid expensive disk lookups for definitely absent keys
- networking and caches

Trade-off

We do not store values, only approximate set membership.

HyperLogLog: Goal

Task

Estimate the number of distinct elements in a huge stream.

Why exact counting is hard

An exact set of all distinct values may be too large to store.

HyperLogLog: Core Intuition

Hash values look random

Rare patterns, such as many leading zeros, are evidence that we have seen many distinct elements.

Main idea

Split the stream into buckets, track a statistic related to the longest run of leading zeros, then aggregate.

Why HyperLogLog Is Popular

Pros

- tiny memory
- mergeable summaries
- good accuracy for large streams

Cons

- approximate only
- not useful when exact identities are needed

Bloom Filters vs HyperLogLog

Bloom filter

Approximate membership.

HyperLogLog

Approximate distinct count.

Big idea

Probabilistic structures answer a smaller question than a full dictionary, but do it at massive scale.

Similarity Search at Scale

Problem

Given a query item, find stored items that are similar to it.

Applications

Near-duplicate documents, recommendation, image retrieval, embedding search.

Challenge

Exact nearest-neighbor search can be too slow in high dimensions.

Locality Sensitive Hashing (LSH)

Main principle

Use hash functions designed so that similar items are more likely to collide than dissimilar items.

Interpretation

Hashing is no longer about exact equality; it becomes a tool for approximate similarity search.

Why LSH Helps

Workflow

- 1 hash all items into buckets
- 2 hash the query
- 3 compare mainly with items in the same or nearby buckets

Effect

Much less brute-force scanning.

Where LSH Fits in the Big Picture

Exact dictionary mindset

Same key means same object.

LSH mindset

Similar objects should land close together.

Trade-off

We gain speed by accepting approximate candidate generation.

Choosing the Right Structure

Ask first

What is the real question?

- exact lookup by key
- ordered lookup
- repeated min extraction
- dynamic connectivity
- approximate membership
- approximate distinct counting
- approximate similarity search

Operational Summary

Need	Typical structure
FIFO processing	queue
LIFO processing	stack
best-priority extraction	priority queue / heap
dynamic connectivity	union-find
exact lookup by key	hash table
exact ordered map	balanced BST
approximate membership	Bloom filter
approximate cardinality	HyperLogLog
similarity candidate search	LSH

What to Remember

- Data structures are about invariants plus operation costs.
- Dynamic arrays need amortized analysis.
- Hash tables give expected constant-time exact lookup, but not sorted order.
- Trees trade extra structure for faster lookup.
- Support structures like queues and heaps shape algorithm behavior.
- Probabilistic structures are essential at modern scale.

Next Time

Coming next

We will use many of these structures inside graph algorithms, network analysis, and spatial search.